

Trabalho Prático — Programação Concorrente e Sistemas Operacionais

Objetivo

Praticar criação e gerência de processos e redirecionamento de E/S em C:

- criar processos com fork;
- conectar processos com pipe;
- substituir imagem de processo com alguma variação de exec (execlp, execvp, etc.);
- sincronizar término com wait/waitpid;
- tratar erros de sistema (boas mensagens e errno).

Especificação

Implemente um programa chamado `pipeline_wc` que recebe 1 argumento (um arquivo texto) e cria um pipeline de dois processos equivalente a:

```
cat <arquivo> | wc -l
```

Requisitos técnicos:

1. Crie um pipe.
2. Crie dois processos com fork:
 - Filho 1 (produtor): redireciona STDOUT para a extremidade de escrita do pipe e executa `cat <arquivo>` via `exec*`.
 - Filho 2 (consumidor): redireciona STDIN para a extremidade de leitura do pipe e executa `wc -l` via `exec*`.
3. O pai fecha todas as extremidades do pipe que não usa e aguarda os dois filhos com `waitpid`, exibindo:
 - o PID e status de saída de cada filho;
 - se algum terminou por sinal, informar qual.
4. Em caso de erro (sistema ou uso incorreto), imprimir mensagem clara e retornar código $\neq 0$.
5. O código de saída do programa deve ser o código de saída do consumidor (`wc`), quando disponível.

Exemplo de uso

```
$ ./pipeline_wc dados.txt
```

```
[PAI] Filho 12345 (cat) saiu com status 0
```

```
[PAI] Filho 12346 (wc -l) saiu com status 0
```

Restrições

- É obrigatório usar fork, pipe, uma variação de exec e wait ou waitpid.
- Proibido usar popen, system ou equivalentes.

Entrega

- Código-fonte pipeline_wc.c
- Pequeno README.md com: instruções de compilação/execução, exemplo de saída e menção às syscalls usadas.

Critérios de Avaliação

- 30% Correção funcional / atende à especificação
- 25% Uso correto de fork/pipe/exec/waitpid e fechamento de descritores
- 20% Tratamento de erros e mensagens claras
- 15% Organização/legibilidade
- 10% Qualidade do README